

Module 7: Production Tracing & Evaluation

Observability, Metrics, and Quality Assurance with LangSmith



Why LangSmith?

As you transition from prototyping chains to deploying complex **LangGraph** architectures in production, visibility becomes your biggest challenge. When an agent fails, halts, or hallucinates, pinpointing the exact point of failure within a multi-turn, multi-tool loop is like finding a needle in a haystack.

LangSmith is a unified DevOps platform designed specifically for LLM applications. It provides fine-grained tracing of every execution step, dataset management for prompt evaluation, and real-time monitoring of token costs and latencies.

Key Philosophy: Without observability, LLM applications are black boxes. LangSmith opens the box, turning subjective output evaluation into a rigorous, data-driven engineering practice.



Configuration & Setup

Integrating LangSmith into any existing LangChain or LangGraph project requires zero code changes—it is controlled entirely via environment variables.

Step 1: Sign up and Generate API Key

Visit smith.langchain.com, create an account, and generate a personal API key.

Step 2: Authenticate Configuration Environment

Add the following variables to your `.env` file. The LangChain framework will automatically detect these and begin streaming telemetry data to your dashboard.

```
LANGCHAIN_TRACING_V2="true"
LANGCHAIN_API_KEY="ls__your_langsmith_api_key"
LANGCHAIN_PROJECT="my_first_agent_project"
```



Core Feature 1: Execution Tracing

Once tracing is enabled, every `invoke()`, `stream()`, or `batch()` call is recorded. A single user request creates a **Run Tree**.

- **Granular Visibility:** See exactly what prompt was sent to the LLM, the exact raw JSON returned, and how long the parsing took.
- **Tool Inputs & Outputs:** If your agent uses a weather tool, LangSmith logs the exact string passed to the tool and the exact string the tool returned.
- **Token & Cost Tracking:** View precise token consumption and estimated dollar costs per run, helping identify inefficient loops.



Core Feature 2: Datasets & Evaluation

How do you know if changing your prompt makes the agent better or worse? You need regression testing. LangSmith allows you to build datasets (Inputs and Expected Outputs) and run evaluators against them.

```

from langsmith import Client
from langsmith.evaluation import evaluate
from langchain_openai import ChatOpenAI

client = Client()

# 1. Create a Dataset programmatically (or via the UI)
dataset_name = "Support_QA_Test"
dataset = client.create_dataset(dataset_name=dataset_name)
client.create_examples(
    inputs=[{"question": "How do I reset my password?"}],
    outputs=[{"answer": "Go to settings > security > reset password."}],
    dataset_id=dataset.id,
)

# 2. Define your application (Target Function)
def predict(inputs: dict):
    llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
    # In reality, this would be your complex LangGraph agent
    return llm.invoke(inputs["question"]).content

# 3. Define an Evaluator (e.g., Exact Match or LLM-as-a-Judge)
def exact_match(run, example):
    predicted = run.outputs["output"]
    expected = example.outputs["answer"]
    return {"key": "exact_match", "score": int(predicted == expected)}

# 4. Run the Evaluation Pipeline
experiment_results = evaluate(
    predict,
    data=dataset_name,
    evaluators=[exact_match],
    experiment_prefix="test-gpt3.5-v1"
)

```

Evaluation Strategies (LLM-as-a-Judge)

Standard software metrics (like Exact Match or BLEU) are often too rigid for generative text. LangSmith integrates with LangChain's evaluation modules to use a superior LLM (like GPT-4) to grade the outputs of a cheaper LLM (like GPT-3.5) based on custom criteria.

Evaluator Type	What it Measures	Use Case
Correctness (QA)		RAG pipelines where factual accuracy is paramount.

	Does the output accurately answer the input question based on the provided reference context?	
Helpfulness / Tone	Is the response polite, clear, and adhering to brand guidelines?	Customer support chatbots.
Toxicity / Bias	Does the output contain harmful or biased language?	Public-facing applications requiring strict safety guardrails.

🔧 Mini Project: Build an Evaluation Pipeline

The RAG Regression Test

Goal: Ensure an update to your retrieval prompt doesn't break existing performance.

Implementation:

1. Use the LangSmith UI to manually select 10 successful traces from your production logs and click "Add to Dataset".
2. Write a custom LangChain agent that performs Document Retrieval.
3. Use the `evaluate()` function with an `LLM-as-a-Judge` evaluator configured to check for "Contextual Hallucination" (ensuring the model didn't invent facts outside the retrieved docs).

✅ Daily Learning Reflection

Use this section to cement your knowledge. Answer these prompts for yourself:

1. Why is traditional print-statement debugging insufficient for complex Agent architectures like LangGraph?

2. Describe the concept of "LLM-as-a-Judge" and when you would use it over traditional metrics like Exact Match.

Generated for technical mastery and structured learning. Keep building!